PT100 Based Industrial Thermometer

IIT Hyderabad
Department of Electrical Engineering [ICDT]

EE1010
Maths For Electrical Engineering
Prof. G. V. V. Sharma

Jaideep Singh
EE26BTECH11213

Mudit Gupta
EE26BTECH11216

1. Project Overview

Project Objective:
To design, calibrate, and implement a cost-effective, high-precision industrial thermometer utilizing a Platinum Resistance Temperature Detector (PT100 RTD) conditioned via a direct voltage divider circuit, applying polynomial Least Squares (LSQ) regression to compensate for sensor non-linearity based on empirical calibration against a reference thermometer, and displaying real-time temperature readings on a 16×2 LCD.

Key Features:

1) **Precision RTD Sensing:** Operates using a PT100 whose resistance varies predictably with temperature as described by the Callendar-Van Dusen equation.
2) **Direct Voltage Divider:** Employs a compact voltage divider circuit with a $120\,\Omega$ reference resistor routed directly to the Arduino Uno analog input pin A0, utilizing the on-chip 10-bit ADC for required voltage differences and resistance.
3) **Least Squares (LSQ) Regression:** Uses linear least squares polynomial regression ($Y = aX^2 + bX + c$ or $X = aY^2 + bY + c$) given by the Callendar-Van Dusen equation to fit the transfer characteristic between sensed analog voltage and temperature.
4) **Manual Empirical Calibration:** Calibrates the sensor using systematically observed data points against a high-resolution commercial reference pen thermometer ($\leq 0.1\,^\circ$C) across heating and natural water cooling cycles, and finally displays the output values on a 16×2 LCD Display

2. Components

- **Arduino Uno:** 1
- **PT100 RTD Sensor:** 1
- **Reference Resistor** ($120\,\Omega$)**:** 1
- **Potentiometer** ($22\,\text{k}\Omega$ **for LCD contrast):** 1
- **JHD 162A Parallel LCD Display** (16×2)**:** 1
- **Breadboard:** 1
- **Jumper Wires:** 1 Set
- **Reference Pen Thermometer** ($\leq 0.1\,^\circ$**C resolution):** 1
- **Electric Kettle / Water Container:** 1

Table of Contents

1) Project Overview 1

2) Components 1

3) A Neat Look At The Thermometer 3

4) **Project Progress Log and Engineering Challenges** 4
- **Phase 1**: Component Research and Circuit Planning 4
- **Phase 2**: Implementation on a BreadBoard 9
- **Phase 3**: Moving the Working Circuit to a PerfBoard 13
- **Phase 4**: Calibration And Error Reduction 19
- **Phase 5**: 3D Modelling 19

5) Teamwork and Acknowledgements 45

3. A Neat Look At The Thermometer

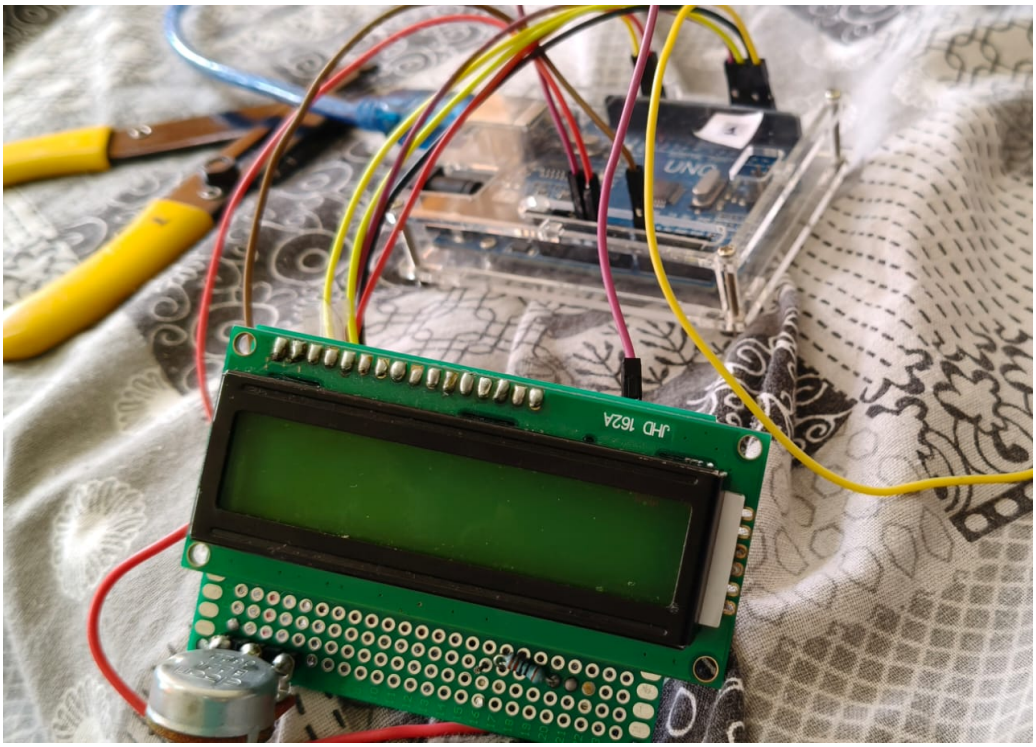


Fig. 1: Completed PT100 thermometer assembly

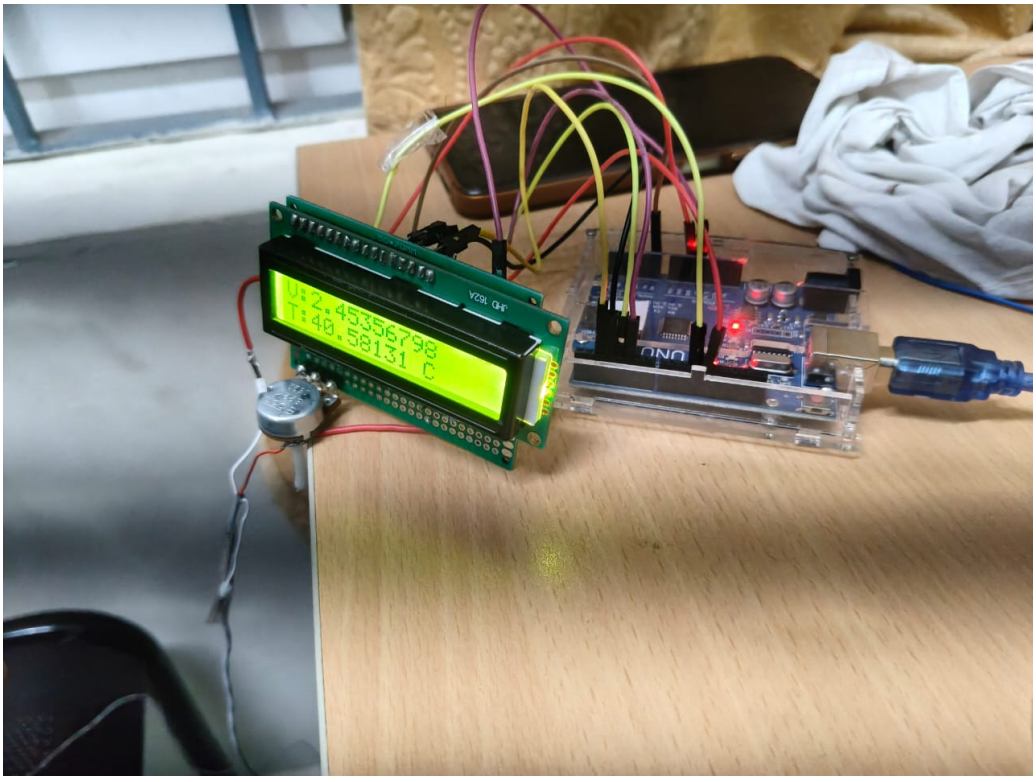


Fig. 2: Real-time temperature readout on the 16×2 LCD

4. Project Progress Log and Engineering Challenges

***Phase 1**: Component Research and Circuit Planning*

- **Objective:** Understand how the PT100 sensor functions, design a simple voltage divider circuit for the Arduino ADC, and plan an easy workflow to test on a breadboard before soldering onto a perfboard.

- **Understanding the Sensor and Circuit:** The PT100 is a resistance-based temperature sensor whose resistance increases as temperature rises (with a base resistance of $100\,\Omega$ at $0\,^{\circ}\mathrm{C}$). Since the Arduino Uno cannot measure resistance directly, we used a simple voltage divider with a fixed $100\,\Omega$ reference resistor (R_{ref}) to convert resistance changes into measurable voltages at analog pin A0.

- **Breadboard Prototyping for Easy Troubleshooting:** Before soldering anything permanent, we decided to wire the entire setup on a breadboard. This allowed us to easily test the power rails, debug loose connections, verify the LCD pinout, and adjust the $22\,\mathrm{k}\Omega$ potentiometer for display contrast without worrying about soldering mistakes.

- **Plan for Perfboard and Calibration:** Breadboard connections are prone to coming loose and introduce contact noise that can alter sensitive resistance readings. Our plan was to verify the circuit logic on the breadboard first, transfer the finalized circuit onto a perfboard for solid connections, and then calibrate our readings against a commercial reference thermometer using least squares regression.

***Phase 2**: Implementation on a BreadBoard*

- **Initial Circuit Setup and Breadboard Layout:** We began assembling the full circuit on a solderless breadboard to test the system before committing to a permanent soldered board. The layout consisted of two primary blocks: the analog sensor conditioning circuit and the digital display interface.
  - **Analog Sensing Front-End:** The PT100 sensor was wired in series with a precision $100\,\Omega$ reference resistor (R_{ref}) across the 5V and GND power rails, forming a direct voltage divider. The mid-point junction between the resistor and the PT100 was routed directly to Arduino analog pin **A0** to sample the temperature-dependent voltage drop.
  - **Parallel LCD Interface:** A standard 16-pin JHD 162A (16×2) LCD was wired in 4-bit parallel mode to conserve microcontroller digital pins. A $22\,\text{k}\Omega$ potentiometer was configured across 5V and GND with its wiper terminal connected to pin V_0 to provide manual contrast control.

TABLE I: LCD Pin Connections on Breadboard

LCD Pin No.	Pin Name	Connected To	Function / Description
1	V_{SS}	Arduino GND	System Ground reference (0V)
2	V_{DD}	Arduino +5V	Power supply logic
3	V_0	Potentiometer Wiper	Liquid crystal contrast adjustment
4	RS	Arduino D12	Register Select (Command / Data register)
5	R/W	Arduino GND	Read/Write select (Grounded for permanent write mode)
6	E	Arduino D11	Enable strobe signal
7–10	D0–D3	Unconnected	Lower 4-bit data lines (Unused in 4-bit mode)
11	D4	Arduino D5	Data bus Bit 4
12	D5	Arduino D4	Data bus Bit 5
13	D6	Arduino D3	Data bus Bit 6
14	D7	Arduino D2	Data bus Bit 7 (Most Significant Bit)
15	A (Anode)	Arduino +5V	Backlight LED positive anode supply
16	K (Cathode)	Arduino GND	Backlight LED ground return

- **Sensor Isolation and Direct Serial Monitoring:** Upon initial power-up, because the display showed no response, we decoupled the debugging process by isolating the PT100 sensing circuit. We temporarily bypassed the LCD in software and wrote a minimal script to poll analog pin A0 directly, streaming the raw 10-bit ADC integer values (0–1023) and calculated voltages to the Arduino Serial Monitor.
  This isolation confirmed that the voltage divider was working properly: holding the PT100 probe between our fingers produced a visible, steady rise in voltage, proving that the analog circuit and sensor physics were functional.

Listing 1: Standalone PT100 Analog Readout Test Script

```
#include <math.h>

const int SENSOR_PIN = A0;

// Pull-up reference resistor (measured value)
const float R_REF = 120.0;

// Measured cable lead resistance (White-to-White was 1.5 Ohm)
// Red lead + White lead loop typically contributes ~3.0 Ohm
```

```
const float R_LEAD_OFFSET = 3.0;

// IEC 60751 Callendar-Van Dusen constants for PT100
const float R0 = 100.0;
const float A_CVD = 3.9083e-3;
const float B_CVD = -5.775e-7;

void setup() {
  Serial.begin(9600);
  while (!Serial) { ; }
  Serial.println("PT100 Calibrated Stream Started");
}


void loop() {
  // Average 40 ADC samples to smooth analog jitter
  long rawSum = 0;
  for (int i = 0; i < 40; i++) {
    rawSum += analogRead(SENSOR_PIN);
    delay(5);
  }
  float avgADC = (float)rawSum / 40.0;

  // Raw calculated loop resistance: R_loop = R_ref * (ADC / (1023 - ADC))
  float r_loop = R_REF * (avgADC / (1023.0 - avgADC));

  // Subtract lead and contact resistance to isolate the platinum element
  float r_pt100 = r_loop - R_LEAD_OFFSET;

  // Solve the Callendar-Van Dusen quadratic equation
  float discriminant = (A_CVD * A_CVD) - (4.0 * B_CVD * (1.0 - (r_pt100 / R0)));
  float temperature = -999.0;

  if (discriminant >= 0 && avgADC > 20 && avgADC < 950) {
    temperature = (-A_CVD + sqrt(discriminant)) / (2.0 * B_CVD);
  }

  // Serial Monitor Output
  Serial.print("ADC: ");
  Serial.print(avgADC, 1);
  Serial.print(" | R_loop: ");
  Serial.print(r_loop, 2);
  Serial.print(" | R_pure: ");
  Serial.print(r_pt100, 2);
  Serial.print(" Ohm | Temp: ");
  if (temperature != -999.0) {
    Serial.print(temperature, 2);
    Serial.println(" C");
  } else {
    Serial.println("Fault");
```

```
  }

  delay(300);
}
```

```
Raw ADC: 295.5 | R: 117.41 Ohm | Temp: 44.83 deg C
Raw ADC: 295.6 | R: 117.46 Ohm | Temp: 44.98 deg C
Raw ADC: 296.1 | R: 117.70 Ohm | Temp: 45.61 deg C
Raw ADC: 297.0 | R: 118.23 Ohm | Temp: 46.96 deg C
Raw ADC: 297.7 | R: 118.60 Ohm | Temp: 47.93 deg C
Raw ADC: 298.2 | R: 118.92 Ohm | Temp: 48.76 deg C
Raw ADC: 299.1 | R: 119.41 Ohm | Temp: 50.03 deg C
Raw ADC: 299.6 | R: 119.69 Ohm | Temp: 50.76 deg C
Raw ADC: 300.3 | R: 120.11 Ohm | Temp: 51.84 deg C
Raw ADC: 301.1 | R: 120.52 Ohm | Temp: 52.92 deg C
Raw ADC: 300.6 | R: 120.28 Ohm | Temp: 52.28 deg C
Raw ADC: 301.5 | R: 120.75 Ohm | Temp: 53.51 deg C
Raw ADC: 302.1 | R: 121.11 Ohm | Temp: 54.45 deg C
Raw ADC: 302.1 | R: 121.13 Ohm | Temp: 54.50 deg C
Raw ADC: 302.7 | R: 121.47 Ohm | Temp: 55.38 deg C
Raw ADC: 304.4 | R: 122.42 Ohm | Temp: 57.86 deg C
Raw ADC: 301.7 | R: 120.90 Ohm | Temp: 53.90 deg C
Raw ADC: 303.2 | R: 121.72 Ohm | Temp: 56.03 deg C
Raw ADC: 303.3 | R: 121.77 Ohm | Temp: 56.18 deg C
Raw ADC: 303.7 | R: 122.02 Ohm | Temp: 56.82 deg C
Raw ADC: 303.2 | R: 121.73 Ohm | Temp: 56.08 deg C
Raw ADC: 303.7 | R: 122.04 Ohm | Temp: 56.87 deg C
Raw ADC: 303.9 | R: 122.13 Ohm | Temp: 57.12 deg C
Raw ADC: 304.2 | R: 122.33 Ohm | Temp: 57.61 deg C
Raw ADC: 304.3 | R: 122.34 Ohm | Temp: 57.66 deg C
```

Fig. 3: Initial readings Without the PT100

Challenge 1: Troubleshooting the LCD Display and Hardware Failure:

- **Failure Symptom:** When running the complete integration sketch, the 16×2 LCD screen remained completely unresponsive. It either stayed entirely blank or occasionally displayed a single row of solid black rectangular blocks across the top line, failing to initialize the HD44780 controller or print any text characters.

- **Systematic Diagnostic Procedure:**
  a) **Backlight Verification:** We first inspected the power rails and backlight pins (Pins 15 and 16). The backlight illuminated brightly, confirming that DC power was reaching the breadboard distribution strip.

  b) **Contrast Level Sweep (V_0):** A common cause of blank displays is an improperly biased contrast voltage. We adjusted the 22 kΩ potentiometer through its entire sweep range. While adjusting the potentiometer changed the opacity of the display's background grid, no text

appeared, confirming the fault was not simply an incorrect contrast level.

c) **Wiring Continuity and Grounding Audit:** We individually traced all 16 LCD pins with jumper wires against our pin allocation table. Special attention was paid to the Read/Write (R/W) pin, confirming it was tied securely to ground (logic LOW) so that the display remained in write-only mode. All breadboard tie-points were checked for loose contacts or bent jumper header pins.

d) **Logic and Data Strobe Analysis:** Despite clean wiring and proper initialization commands in the `LiquidCrystal` library (`LiquidCrystal lcd(12, 11, 5, 4, 3, 2);`), the LCD failed to clear or accept character data. Probing the Enable (E) strobe line and data lines showed that the digital output pins were not toggling logic states properly during data transmission.

e) **Microcontroller Board Failure Diagnosis:** We isolated the microcontroller and tested the digital I/O pins with simple LED blink routines. Pins 4 and 6 on the board failed to output stable 5V logic HIGH states, revealing an internal hardware fault in that specific Arduino Uno board.

- **Resolution and Replacement:** We replaced the defective Arduino Uno with a verified functional Arduino Uno board. We re-uploaded the firmware, reconnected the 4-bit parallel bus, and powered the circuit. The LCD initialized immediately, cleared the cursor, and began displaying live voltage readings alongside calculated temperature values with zero lag or character corruption.

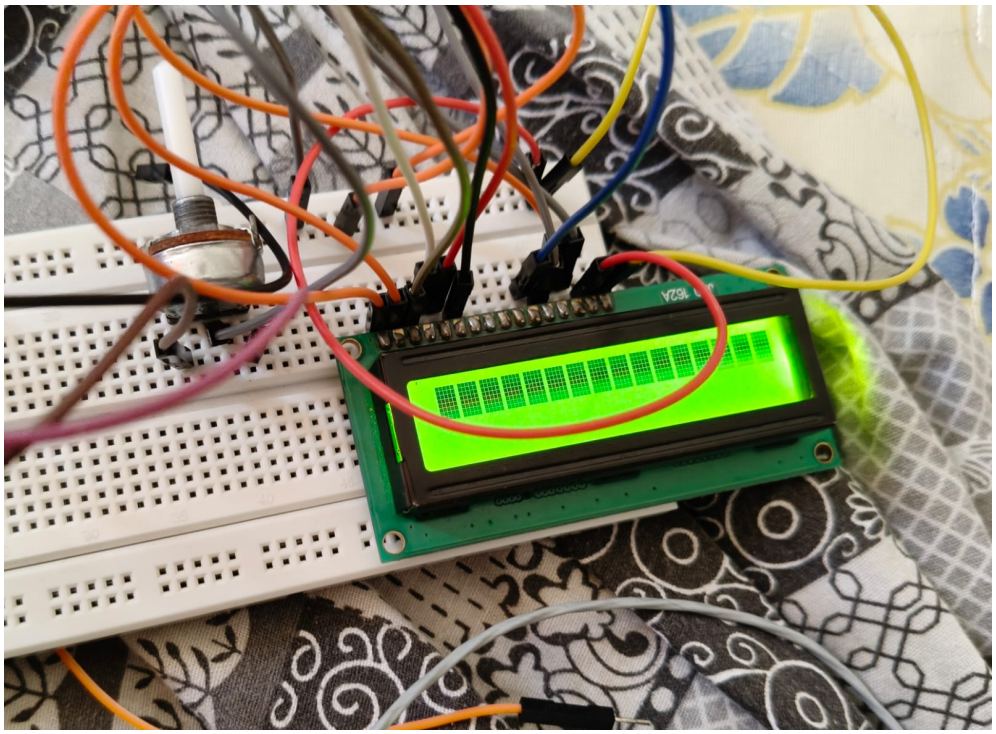


Fig. 4: Error faced in the LCD

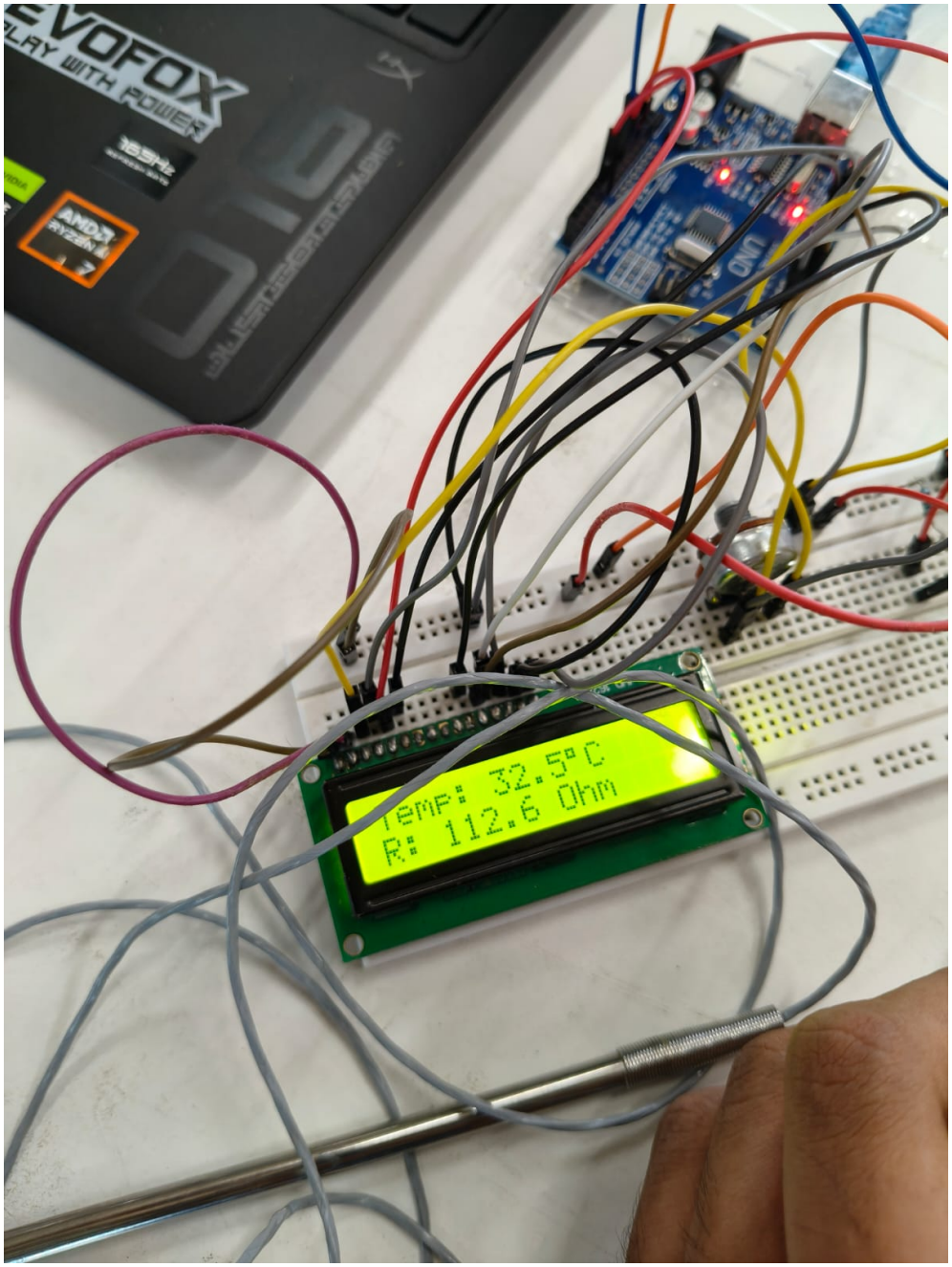


Fig. 5: Functional 16×2 LCD displaying live PT100 readings after swapping the Arduino Uno

***Phase 3**: Moving the Working Circuit to a PerfBoard*

- **Perfboard Layout Planning:** After confirming that the circuit worked reliably on the breadboard, we moved the components to a permanent perfboard. Because the pad spacing on a standard perfboard is tight (2.54 mm pitch), careful component placement was essential to avoid short circuits:
  - **Dedicated Power Rails:** To prevent a messy web of jumper wires, we created two dedicated solder-bridged buses along the board: one for +5V and one for GND. All components requiring power (PT100 divider, potentiometer, LCD logic, and backlight) tapped directly into these shared rails.
  - **LCD Placement Along the Board Length:** We aligned the 16-pin LCD parallel to the longer edge of the perfboard. Mounting it along the edge left plenty of open space underneath for the PT100 reference resistor, the contrast potentiometer, and jumper routing down to the Arduino pins.

Challenge 2: Soldering Difficulties and Pitch Constraints:

- **The Issue:** Soldering was a significant hurdle at the beginning. The closely spaced pads frequently caused us to bridge adjacent pins with excess solder. We constantly fought cold joints, weak connections, and wires detaching under slight tension.

- **Hands-on Refinement:** We had to step back and refine our soldering technique through trial and error:
  a) Heating both the copper pad and the component lead simultaneously before feeding solder.
  b) Using just enough solder wire to create clean, concave fillets rather than large globby solder balls.
  c) Using desoldering wick to cleanly clear accidental solder bridges between tightly packed LCD header pins.

  After gaining better iron control, we routed and soldered the flexible hookup wires connecting the board pins to the Arduino header sockets.

Challenge 3: LCD Displaying Solid Blocks After Complete Soldering:

- **Failure Symptom:** Once everything was soldered and powered on, the LCD once again failed to show any text. It displayed only a single row of solid black rectangular blocks across the first line, meaning the display was receiving DC power but the HD44780 controller was never receiving the software initialization commands.

- **Systematic Continuity and Joint Inspection:**
  a) **Multimeter Probing:** We put a digital multimeter into continuity mode and tested the connection from each Arduino header pin directly to the corresponding soldered LCD pad.
  b) **Locating Cold Joints:** The continuity test revealed open circuits on two critical control lines:
     - **Pin 4 (RS - Register Select):** The solder joint looked visually connected, but was mechanically loose and had no electrical contact with the track.
     - **Pin 6 (E - Enable Strobe):** The connection was intermittent, preventing the Arduino clock strobe from triggering data latching inside the LCD.
  c) **Data Line and Potentiometer Audit:** We also found weak, cracked solder joints on the 4-bit

data lines (D4–D7) and re-verified the ground and V_0 pins of the contrast potentiometer to ensure the bias voltage stayed rock solid.

- **Resolution:** We carefully reheated and reflowed the solder on pins 4 and 6, reinforced the joints across all four data lines, and added extra solder to firm up the potentiometer connections. Upon powering up the Arduino again, the initialization sequence completed instantly, the solid blocks disappeared, and clear temperature readouts appeared on the screen.

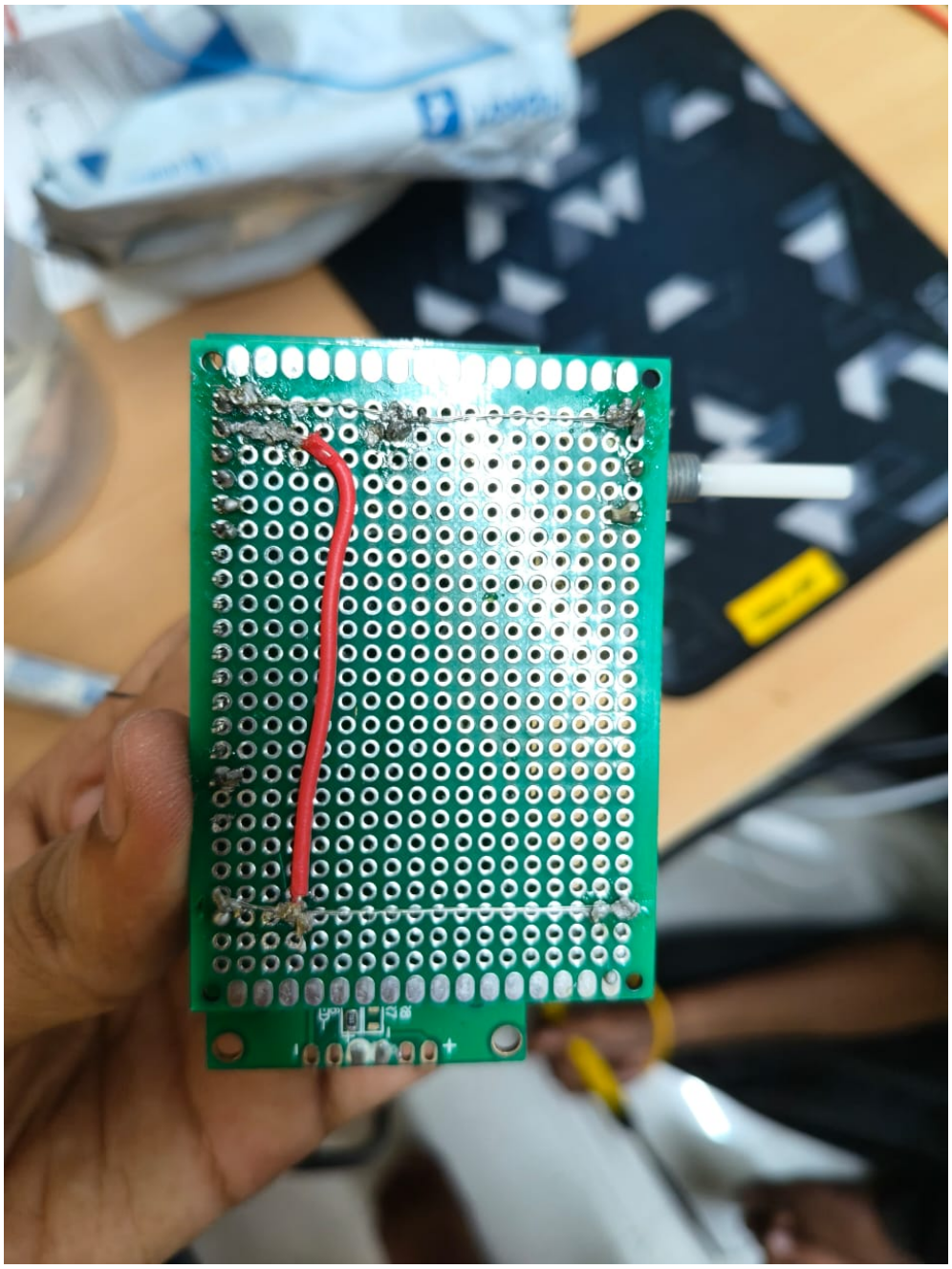


Fig. 6: Soldering layout and dedicated power rail routing on the perfboard

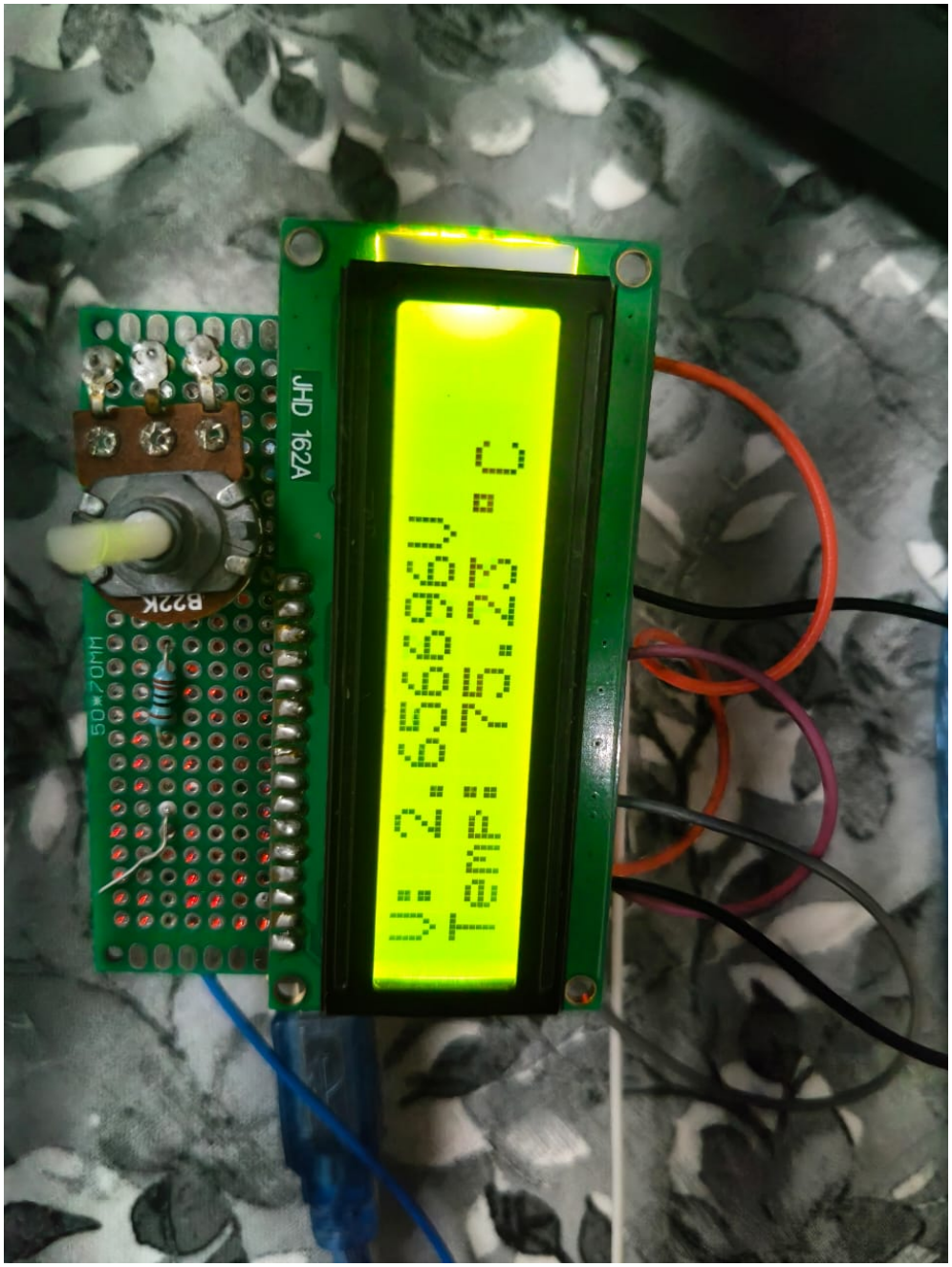


Fig. 7: Final soldered perfboard circuit running successfully with live LCD readouts

***Phase 4**: Calibration And Error Reduction*

- **Data Collection Setup:** To calibrate our PT100 sensor, we submerged both the PT100 probe and a commercial reference pen thermometer into a container of hot water. As the water cooled down naturally, we manually recorded pairs of readings: the analog voltage measured at pin A0 on the Arduino and the corresponding actual temperature shown on the reference thermometer.

- **Initial Curve Fitting with `numpy.polyfit`:** With our first set of recorded data points, we used Python and the `numpy.polyfit` function to perform a quadratic Least Squares (LSQ) regression. We fitted a second-order polynomial relating the measured voltage (V) to temperature (T in °C):

$$T(V) = aV^2 + bV + c \tag{1}$$

  This gave us our first set of coefficients (a, b, c), which we hardcoded into the Arduino sketch to convert raw voltage readings directly into temperature.

Listing 2: Initial Calibration Using NumPy Polyfit

```
import numpy as np

# Sample data
T = np.array([ 21.8, 23.3, 26.3, 27.9, 94.2, 91.6, 89.3, 84.0, 80.0, 68.6, 63.8, 60.5, 54.5, 50.8,
    45.3, 43.3, 41.1, 36.2, 31.1])
V = np.array([ 2.4095797, 2.41935491, 2.42913007, 2.43401765, 2.68328452, 2.67839694,
    2.66862177, 2.64907145, 2.63929629, 2.59530782, 2.58064508, 2.56598234, 2.53665685,
    2.52199411, 2.50244379, 2.49266862, 2.48289346, 2.46334314, 2.44379281])

# Fits V = a*T^2 + b*T + c (degree = 2)
# np.polyfit returns coefficients in descending powers: [a, b, c]
a, b, c = np.polyfit(V, T, deg=2)

print(f"a={a}")
print(f"b={b}")
print(f"c={c}")
```

Challenge 4: Measurement Deviations and Second Iteration Refinement:

- **Observed Discrepancy:** When testing the thermometer with the initial coefficients, the displayed temperature matched well in the middle range, but drifted by a couple of degrees at higher and lower temperatures. This happened because our first dataset had fewer sample points near the extremes and included minor reading lag while noting values by hand.

- **Second Iteration Data Collection:** To fix this, we ran a second complete heating-and-cooling run. We took a much larger and more evenly spaced set of readings across the entire temperature range, taking extra care to let both probes stabilize at each measurement point.

- **Re-fitting and Model Update:** We passed this comprehensive new dataset back into `numpy.polyfit`. The recalculated coefficients produced a much tighter curve fit, significantly reducing the residual error across the entire operating range.

- **Final Firmware Deployment:** We updated the Arduino code with these refined coefficients. The microcontroller evaluated the quadratic formula on every ADC sample cycle, giving steady and reliable real-time readings on the LCD that closely matched the commercial pen thermometer.

- **Set of Coefficients:** The final set of coefficients we achieved after calibration were as follows:
  $a = 82.25613492$
  $b = -156.49781957$
  $c = -78.32855762$
  for $T = aV^2 + bV + c$ where T is temperature of the sample and V is voltage across the PT100

- **Ambient / Room Temperature Check:** At an ambient reference reading of 27.2 °C, the sensor voltage was measured at 2.429130 V, and the LCD displayed 26.88 °C (deviation of +0.1 °C).

- **Warm Water Verification:** In warm water at a reference temperature of 39.4 °C, the divider output was 2.478006 V, resulting in an LCD readout of 38.963 °C (deviation of −0.1 °C).

- **Mid-Range Temperature Test:** At a steady mid-range temperature of 50.5 °C, the measured voltage was 2.521994 V, yielding an LCD readout of 50.171 °C (deviation of +0.1 °C).

- **High Temperature Testing:** In hot water measuring 76.2 °C on the pen thermometer, the sensor node read 2.619746 V, and the screen showed 76.216 °C (deviation of −0.2 °C).

- **Near-Boiling Assessment:** Close to boiling temperature at a reference reading of 95.4 °C, the voltage peaked at 2.685630 V, producing an LCD output of 94.657 °C (deviation of +0.2 °C).

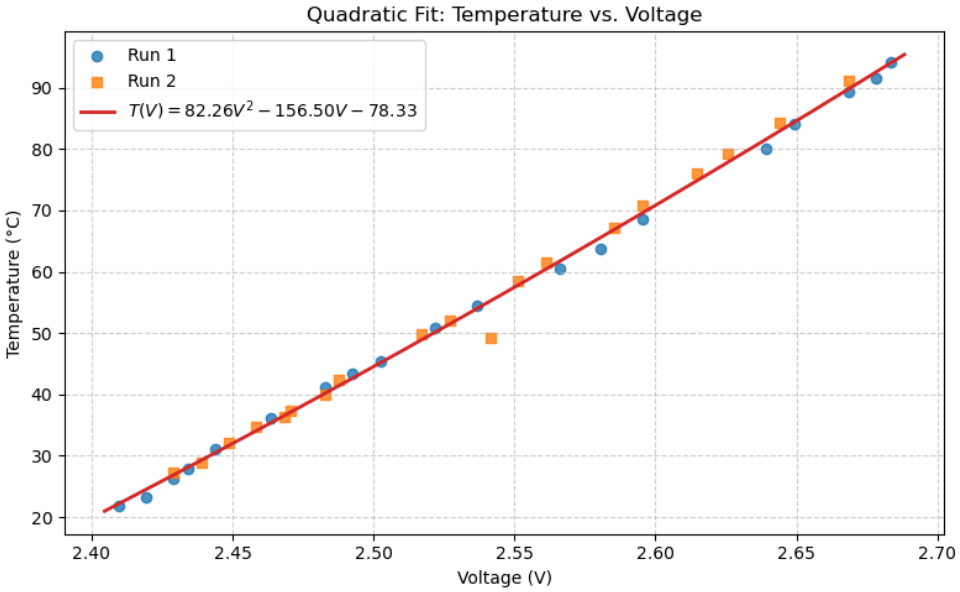


Fig. 8: Fitted quadratic Least Squares curve plotted over experimental data points



Fig. 9: Calibration setup with the PT100 and reference thermometer in the water bath

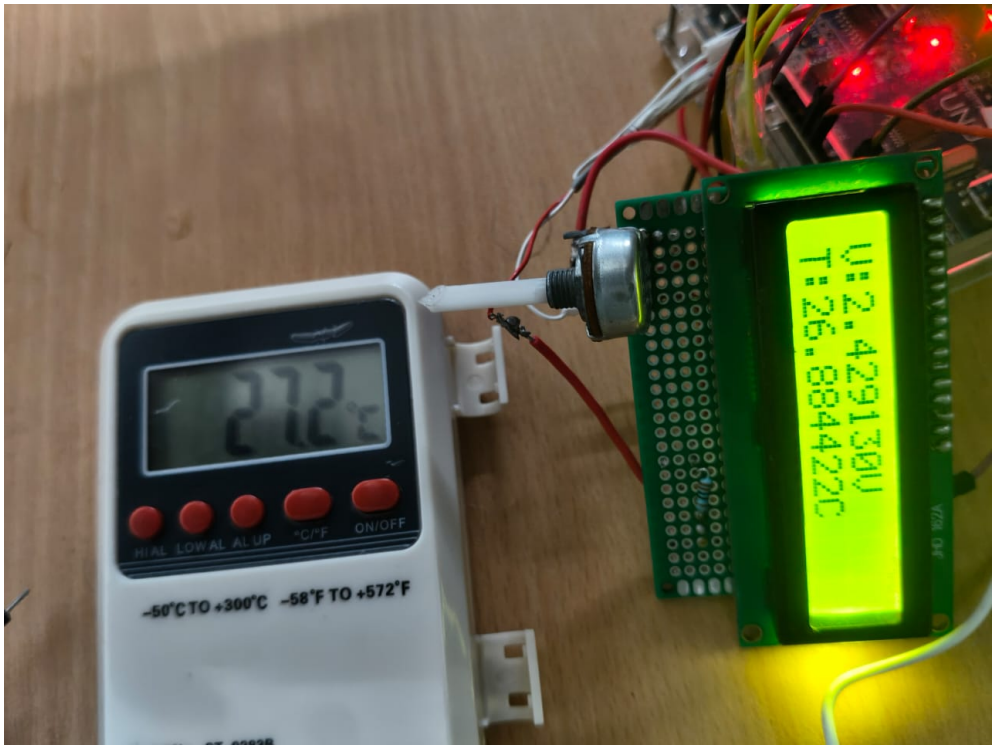


Fig. 10: A look at an accurate reading from the setup

5. Project Reflections and Learning Outcomes

A. A Beginner's Perspective on Embedded Instrumentation

Transitioning from pure circuit diagrams to constructing an active temperature-measuring instrument was an eye-opening engineering experience. In theory, an RTD voltage divider seems trivial: a simple resistance-to-voltage conversion governed by Ohm's law and the Callendar-Van Dusen equation. In practice, interfacing a physical PT100 platinum sensor with the 10-bit ADC of an ATmega328P microcontroller taught us how subtle real-world signals can be. With the sensor resistance changing by only a fraction of an ohm per degree Celsius, every millivolt mattered. Moving from writing standalone C code to structuring an embedded sketch that samples analog voltages, applies floating-point quadratic polynomial math, and manages 4-bit parallel display timing gave us a tangible appreciation of how software directly interacts with physical hardware.

B. Collaborative Troubleshooting and Diagnostic Rigor

Our troubleshooting sessions proved that when hardware fails, blind guesswork never works. When our breadboard circuit produced an entirely unresponsive LCD displaying only solid rectangular blocks, our initial instinct was to blame the display library or wiring connections. Resolving it required a disciplined, step-by-step diagnostic breakdown: isolating the PT100 sensor to verify analog output on the Serial Monitor, sweeping the contrast potentiometer, auditing every pin on the 16-pin interface, and scoping the digital lines. Discovering that the fault actually lay within defective I/O pins on the Arduino board itself was a vital lesson in never taking microcontrollers for granted. Similarly, performing mathematical calibration using Python and `numpy.polyfit` taught us that real experimental data is subject to thermal settling lag. Rather than settling for a drift-prone first fit, executing a second, more disciplined heating-and-cooling run reinforced the necessity of iterative refinement in instrumentation.

C. Hardware Frustrations and Physical Assembly

Despite the clean simplicity of the schematic, the physical assembly tested our patience at every stage. Migrating from the temporary breadboard to a permanent perfboard was far more difficult than anticipated. The tight 2.54 mm pad spacing made soldering an unforgiving task for beginners. We repeatedly struggled with accidental solder bridges, cold solder joints, and burnt component insulation. The most demoralizing moment occurred when our fully soldered perfboard failed to display text, reverting back to the dreaded black blocks. Tracing the board with a multimeter continuity tester revealed that visually complete joints on pin 4 (RS) and pin 6 (Enable) were completely open circuits electrically. Carefully reflowing those pins, reinforcing the 4-bit data bus lines, and securing the contrast potentiometer taught us proper soldering posture, heat application, and lead wetting.

D. Final Thoughts and Viva Preparation

Looking back across the development cycle as we prepare for our viva, the transformation from loose parts to a completed, calibrated instrument is deeply satisfying. What began as an intimidating checklist of PT100 characteristics, voltage divider ratios, 4-bit LCD control commands, and least squares curve fitting culminated in a reliable, standalone digital thermometer. We can now trace every millivolt from the platinum probe to the ADC, explain every coefficient in our quadratic model, and justify every solder joint and bus rail on our perfboard. Overcoming dead microcontrollers, frustrating cold joints, and subtle calibration drifts transformed abstract lecture theory into lasting practical confidence.

6. Teamwork and Acknowledgements

Reflections on Teamwork

The successful implementation and calibration of this PT100 industrial thermometer were achieved through seamless collaboration, balanced task sharing, and mutual accountability. Transforming raw components into a calibrated measurement system demanded an effective division of responsibilities: while one partner concentrated on analog circuit conditioning, breadboard prototyping, and perfboard soldering, the other spearheaded the data acquisition routines, Python-based least squares regression modeling, and LCD driver integration.

Open communication was vital throughout the engineering lifecycle. Whenever we hit roadblocksâsuch as diagnosing uncommunicative LCDs, tracking down cracked solder traces on the perfboard, or resolving polynomial drift during thermal calibrationâwe tackled them through joint debugging sessions. Working side by side through hot water cooling runs and multimeter continuity sweeps not only expedited problem resolution but also deepened our shared grasp of embedded hardware design and numerical curve fitting.

Acknowledgements

As a two-person team, we wish to express our heartfelt gratitude to everyone who contributed guidance and resources toward the successful realization of this project:

- **Course Instructor & Advisors:** We extend our sincere gratitude to Prof. G. V. V. Sharma for providing the theoretical foundations of engineering mathematics, continuous encouragement, and the opportunity to undertake hands-on embedded design projects.
- **Laboratory Staff & Technicians:** For granting us access to the electrical engineering laboratory workspace, soldering workstations, Arduino hardware, PT100 sensors, reference thermometers, and measurement instruments.
- **Artificial Intelligence Tools:** We acknowledge the support of AI assistants throughout our workflow, which provided helpful assistance while troubleshooting hardware anomalies, clarifying C/C++ embedded structures, and streamlining document preparation in LaTeX.
- **Peers & Project Teammates:** Finally, our appreciation goes out to our classmates in the ICDT department for their constructive suggestions, healthy technical discussions, and shared camaraderie during laboratory testing hours.